

Lab 4 – Sequential circuits in Verilog

Learning Outcomes

Lab 4 teaches you:

- how to design different types of counters and timers;
- how to use the Modelsim simulator to verify the correct function of your design and the use of testbenches;
- how to predict the maximum operating clock frequency of your clocked circuits;
- how to design some useful timing and counting components for later Laboratory Sessions

Reminder of Steps to create a design for DE10-Lite FPGA board

1. Create a project using Project Wizard and name convention labXtaskY and top-level Verilog file as **labXtaskY.v**.
2. Specify the Device used as: **10M50DAF484C7G**.
3. Edit the Quartus Setting File (.qsf) and insert pin assignment file **pin_assignment.txt**.
4. Create or edit the different Verilog specification files (.v).
5. Set top-level entity to be **labXtaskY.v** with **Project -> Set Top-Level Entity**
6. Add other components (.v) to the project with **Project -> Add/Remove Files**
7. Specify **timing constraints** in design constraint file (.sdc) file (e.g. clock frequency) – not strictly required.
8. Compile and synthesize the design.
9. Program the DE10-Lite with the bit-stream file (.sof).

Task 1: Simulating a 4-bit counter

Step 1: Create the project for a 4-bit counter

- Create in your work directory a folder named lab4 and in that, another folder task1.
- Click **file>New Project Wizard**, and create project **lab4task1** and top level file **lab4task1**. Then click **Finish**. (Quartus create a database structure in the folder: <home>/lab4/task1 with a project named lab4task1.
- Create the Verilog file: **“counter_4.v”** which contains your design in Verilog. I suggest you use convention of using **“_n”** to indicate the number of bits in a module.
- Click **File > New ...** and select Verilog as the new file. An edit window will appear.

Step 2: Enter the Verilog specification of the 4-bit binary counter

- Enter the Verilog module as shown below (next page). Although you can miss out the comments, I recommend that you retain them because the code is deliberately verbose in order to explain the meaning of the Verilog language.
- The line **timescale 1ns / 100ps** tells the system to use 1 ns as the unit time step with a time resolution of 100ps. (Note first character is special – it is NOT the apostrophe ‘ but the ‘backwards apostrophe `.)
- Make sure that you fully understand this Verilog code before proceeding to the next step. Save the file as **counter_4.v**. (I recommend that you use module name as the file name to avoid confusion.)

Step 3: Compile for Simulation only

- While counter_4.v is opened in the Editor window, click **Project > Set as Top Level Entity**. This command tells Quartus that this module is the top-level of your design.

Normally we use ..._labXtaskY.v as the top-level module, which connects to physical pins on the FPGA. However, for this experiment, the counter module is verified through simulation. So we don't need to create pin connections. The "Set as Top Level Entity" is very useful if you want to use the simulator to verify different component modules in a large design. You can move up and down the module hierarchy and verify them from the lowest level up to the chip level.

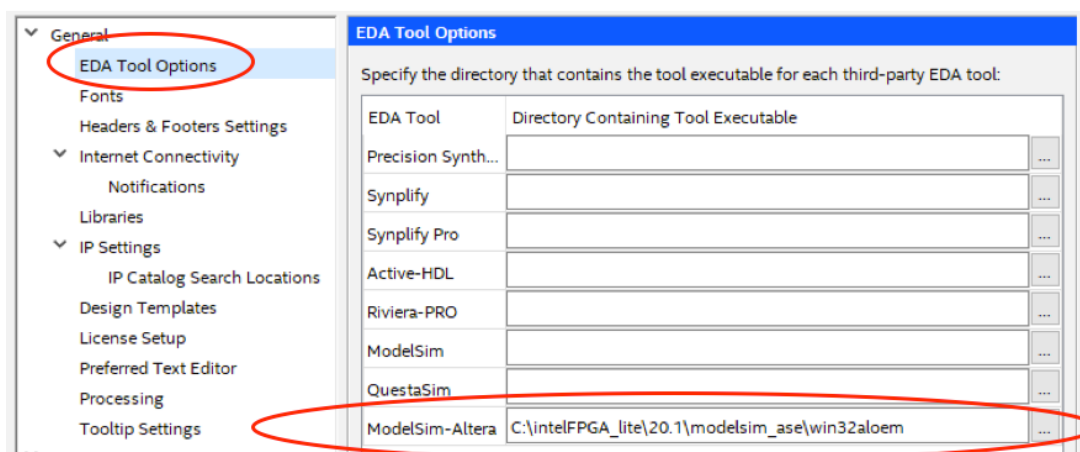
```
1 timescale 1ns / 100ps // unit time is 1ns, resoluti
2 //-----
3 // Design Name: counter_4
4 // Function : a 4-bit synchronous counter with synchron
5 //-----
6
7 module counter_4 (
8     clock, // clock input
9     enable, // high enable counter
10    reset, // high reset counter to 0
11    count // count value
12);
13
14 //----- Declare ports -----
15
16 parameter BIT_SZ = 4;
17 input clock;
18 input enable;
19 input reset;
20 output [BIT_SZ-1:0] count;
21
22 // count needs to be declared as reg
23 reg [BIT_SZ-1:0] count;
24
25 //---- always initialise storage elements such as D-FF
26 initial count = 0;
27
28 //---- Main body of the module -----
29
30 always @ (posedge clock)
31     if (reset==1'b1)
32         count <= 0;
33     else if (enable == 1'b1)
34         count <= count + 1'b1;
35
36 endmodule // end of module
```

Note: A red circle highlights the backslash character in the timescale line, with a callout stating: '\ is a backquote not a ''

- Click **Processing > Analyze Current File**. This is the fastest way to check if this .v file has any syntax error.
- Then Click **Processing > Start > Start Analysis and Synthesis**. This takes the current Verilog module (and all other modules that it uses if any), and produce a register-level model of your design ready for register-transfer level (RTL) simulation. Unlike full compilation, this step does not require pin assignment and other device specific steps, but is sufficient for you to simulate the circuit as specified in Verilog.

Step 4: Set up your PC or MacBook for ModelSim

For Windows 10 users: You need to tell Quartus where the simulator ModelSim is located. Click **Tools > Options**. In the popup window specify the path for the modelsim program as shown below.



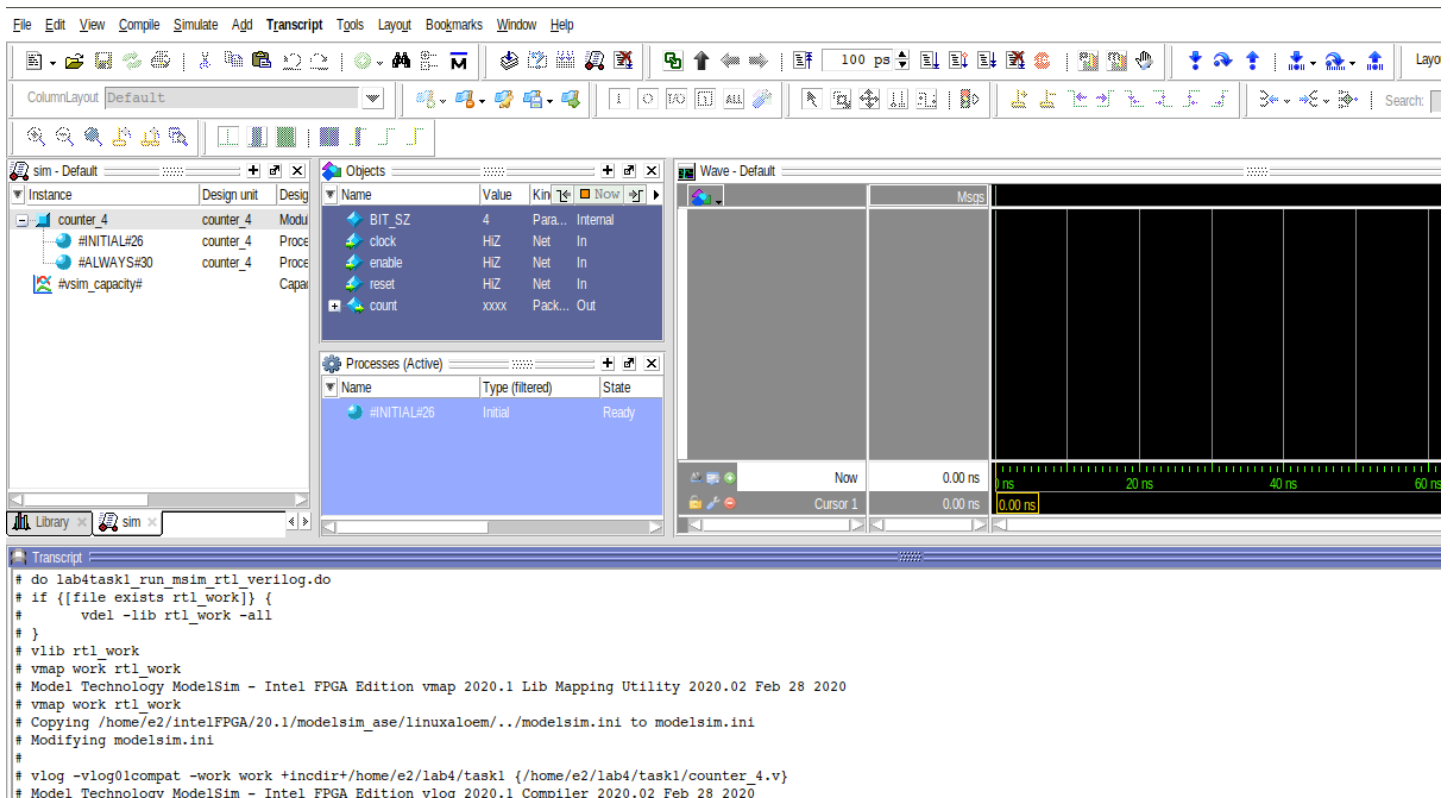
For VirtualBox and Ubuntu users: You need to open a command window and enter the following:

```
sudo apt-get install g++-multilib
```

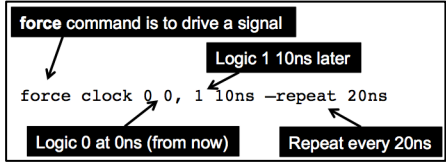
This allows the 32-bit ModelSim simulator include with Quartus Prime Lite to work with the 64-bit Ubuntu.

Step 5: Simulate the binary counter

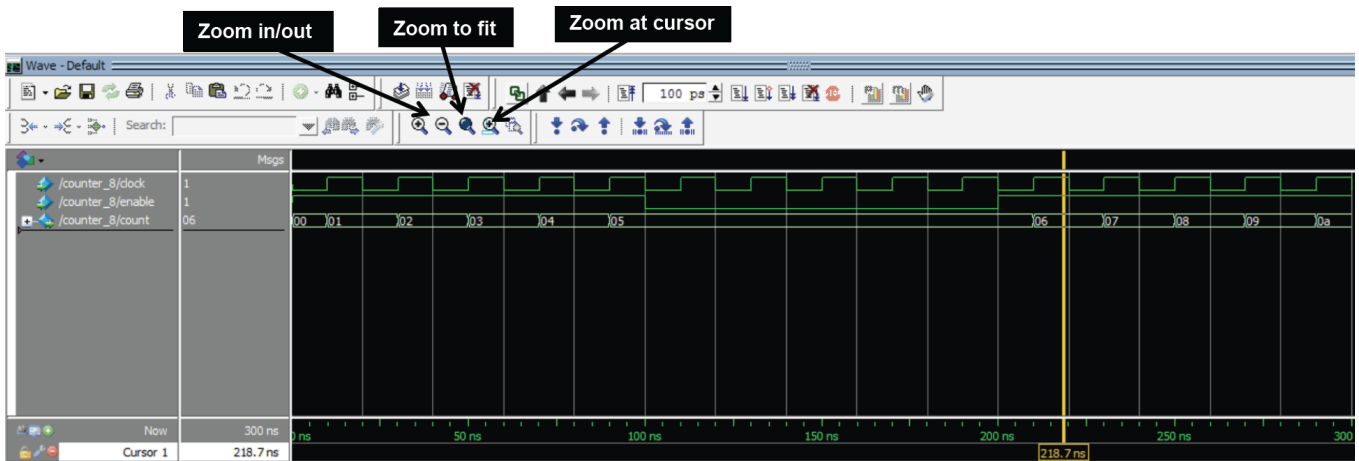
- Click **Tools > Run Simulation Tools > RTL Simulation**. This command starts up Modelsim simulator programme as a separate process. Now you have entered the Modelsim environment.
- Click **Simulate > Start Simulation** Then select **work -> counter_4** from the popup window. This tells Modelsim to simulate the counter_4 module.
- Note that Modelsim provides several windowpanes. The most important is the Transcript pane – this is where you enter commands¹ to drive the simulator. The wave pane is where results are displayed as waveforms. Finally, there is the object pane, which shows all the signals (objects) of your design. You can arrange the different window pane by adjusting the boundaries to suite your preferences.



Step 6: Add waveforms to the Wave window and drive signals

- In the transcript window, enter two commands: **“add wave clock enable reset”** and **“add wave –h count”**. This will add these signals as waveforms in the wave pane and show count values as hexadecimal.
- Now we want to drive **clock** with a 50MHz symmetrical signal (period = 20ns). To do this, enter:

- Enter: **“force enable 1”** to enable the counter.
- Enter **“force reset 0”** to avoid resetting the counter.
- Enter: **“run 100ns”** to run the simulator for 5 clock cycles (5 x 20ns = 100ns).
- You will see the waveform pane showing the counter counting from 0 to 5. Now force enable low and run for another 100ns. Then high again and run for 100ns.

¹ Modelsim uses a scripting language known as **Tcl** to control how it is driven. You only need to learn Tcl if you want to do advance stuff with Modelsim for your personal interest.



- Click on the waveform put a cursor at a specific time for inspecting the signal values. The icons above the waveforms (as labeled) allow you to zoom in and out of the waveform. Try this yourself.

Step 7: Create a Testbench as a DO-file

- Interactively specifying the driving signals is very tedious and prone to error. Therefore the preferred method is to create a “do” file which is a text file containing a sequence of commands (as you have previously entered in the transcript window).
- Click **File > new > source** and select new “do” file. Then enter the command lines as shown on the right. Then save this as “**tb_counter.do**”.
- Delete all signals from the wave window, and enter command

```
vsim> restart
vsim > do tb_counter.do
```

```
/home/e2/lab4/task1/simulation/modelsim/tb_counter.do - Default *
Ln#
1  add wave clock enable reset
2  add wave -hexadecimal count
3  force clock 0 0, 1 10ns -repeat 20ns
4  force reset 0
5  force enable 1
6  run 100ns
7  force enable 0
8  run 100ns
9  force reset 1
10 run 100ns
11 force reset 0
12 force enable 1
13 run 600ns
14
```

- The .do file can be reused and modified far easier than typing them into the transcript window. It acts as a simple form of a **test-harness** (or **testbench**) for your design. Generally speaking, you must produce testbenches for all your designs instead of using typing commands to drive your circuits. Not only because this saves time, it also allows you to change the Verilog design and verify its correctness in the same way for each version.
- Make sure you understand how each line in the DO file results in what you see in the waveform.

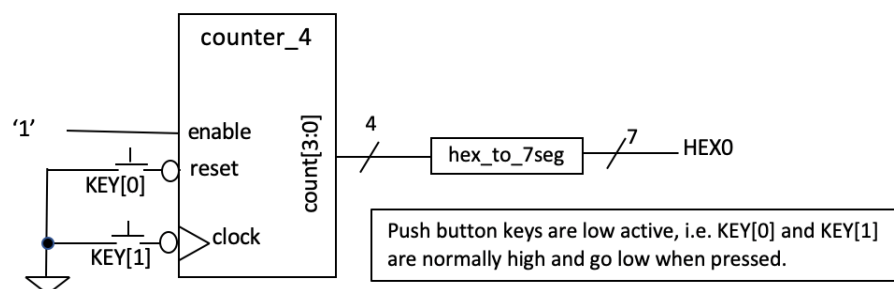
Step 8: Single stepping

- Modelsim is very powerful. You can use it to debug your Verilog design almost like software. However, do remember that we are dealing with a hardware description that operates in parallel. In contrast, software codes are generally procedural and operate sequentially.
- Try the **vsim> step** command or click on the step-command pane  to watch how you can step through your Verilog code. Signal values in the object and the wave windows are updated accordingly.
- Modelsim has many useful features to help you debug your design. Details of all the commands can be found in the Modelsim Reference Manual. This is easily available

under **Help > PDF Documentations > Reference Manual**. Beware that this manual is very thick! DO NOT print this out.

Test yourself – putting the 4-bit counter on DE10-Lite Board

- Download the solution: lab4task1_sol.sof (zipped) from the course webpage and see what you are trying to achieve.
- Create the top level design file: **lab4task1.v** that corresponds to the circuit shown below. Note that the clock of the counter is drive by the momentary switch KEY[1] and not the 50MHz system clock. (This is a good test to see that you can create a complete design and implement on the DE10 board. Refer to the steps on page 1 to produce a working bit-stream file.)
- Why pressing the reset key does not reset the counter? What else do you need to do to perform a reset?



Task 2: 16-bit counter clocked by 50MHz system clock

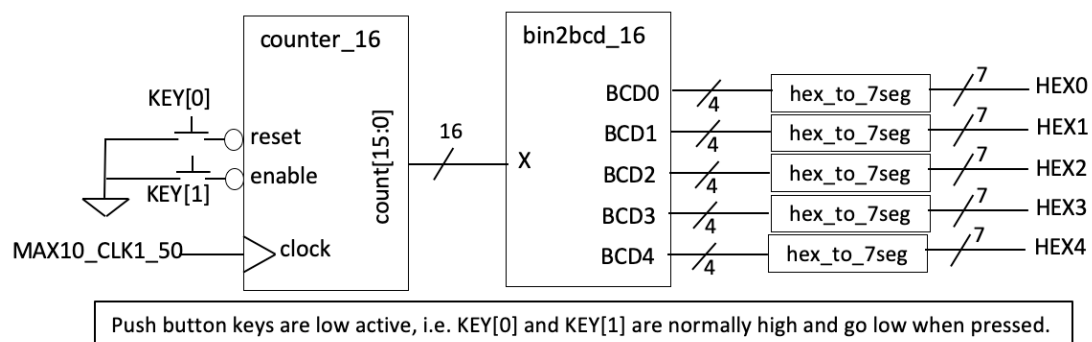
In this part of the experiment, you will implement a 16-bit counter and display its count value as a binary code decimal (BCD) number on the 7-segment displays. You will also learn how to find the maximum clock frequency at which your design will work correctly.

Step 1: Create a new project **lab4task2**, and copy to this directly your **counter_4.v**. Modify **counter_4.v** to **counter_16.v** and make it a 16-bit counter. Download from the experiment webpage the component **bin2bcd_16.v**, a module I have designed to convert a 16-bit binary number to 5 BCD digits. Put these module in the **<home>/mylib** folder, which should also contained the **hex_to_7seg.v** you designed in Lab 3 and the **pin_assignment.txt** file. This is the repository for all the common components/modules that you have designed and will be reused later.

Step 2: Selecting the FPGA Device – Click **Assignments > Device....** and select the correct MAX10 FPGA: **10M50DAF484C7G**.

Step 3: Pin Assignment – Open the **lab4task2.qsf** file (Quartus setting file). Examine its content to make sure that the device and top-level entity is correctly specified. You will find that no pins are being assigned yet. Insert into this file all the pin assignments information. The easiest way to do this is to go the bottom of the file, click on: **Edit > Insert file ..** then select **<home>/mylib/pin_assignment.txt** (you should have downloaded this file from the Experiment webpage and place this in **mylib**). Note that you are currently not using all the pins assigned in the **pin_assignment.txt** file. Don't worry – all used pins are ignored. This will produce a few more warning messages but full compilation can still go ahead without errors.

Step 4: Create a top-level module **lab4task2.v** in Verilog to specify the circuit shown below. Go to the **lab4task2.v** window and set this file as your top-level module. Make sure that you have added all the relevant Verilog modules to the project using **Project > Add/Remove Files in Project: counter_16.v**, and add **hex_to_7seg.v**, and **bin2bcd_16.v** from your library folder **<home>/mylib/**.



Step 5: Use **Processing > Analyze Current File** check your newly create Verilog files. This is the quickest way to find the basic syntax errors in your Verilog code. Once all the simple errors are fixed, use **Processing > Start Analysis and Elaboration** to perform fuller check of the **"lab4task2.v"** to make sure that files are consistent and correct. There is no need to simulate this circuit.

Step 6: Set clock frequency – Create a new file “lab4task2.sdc”² which should contain one single line:

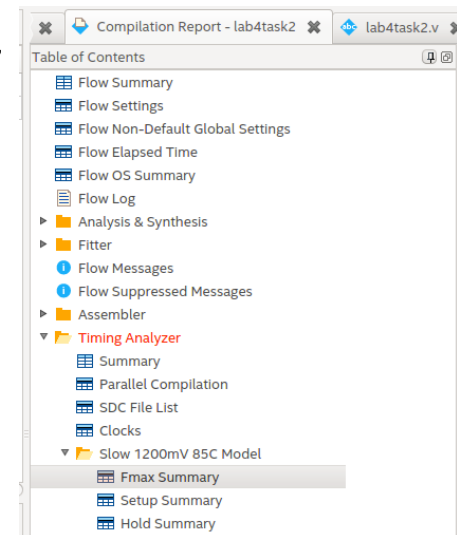
```
create_clock -name "MAX10_CLK1_50" -period 20.000ns [get_ports {MAX10_CLK1_50}]
```

With this, Quartus will know that the signal MAX10_CLK1_50 is a 50 MHz clock signal.

Step 7: Full Compilation – Click: **Processing > Start Compilation**. This will go through the entire compilation process. Examine the Tasks window on the left and see all the steps being taken in order to generate the final bit-stream.

Step 8: Maximum clock frequency – As part of the compilation process, **TimeQuest** timing analyzer is used to predict various timing information. In the “Compilation Report” window, you should see a list of reports resulting from the compilation. Double-click **TimeQuest Timing Analyzer** entry, and you should see a list similar to the one shown here. Clicking on various entries under this will show the various timing specifications. Answer the following questions:

What are the predicted maximum frequencies for this circuit under the highest and lowest temperatures? What are the other interesting timing data that you can discover with these reports? Why is the TimeQuest entry red, indicating that there may be a problem?



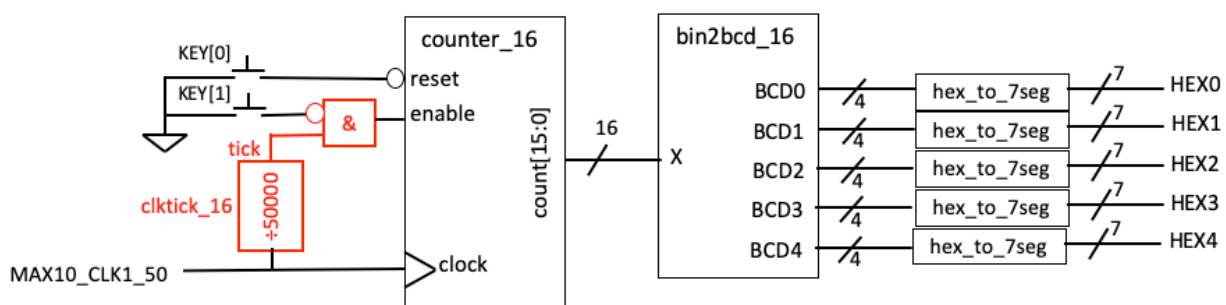
Step 9: Test your design on the DE10 – program the DE10 and check that your design works.

Step 10: Examine the amount of FPGA resources being used by this 16-bit counter. Explain the results.

Test-yourself Task – Cascade counter

You are now required to create something yourself. In the previous exercise, the 16-bit counter is counting with a 50MHz clock. This is much too fast for us to see the counter changing. This part of the experiment requires you use the counter to count the number of millisecond elapsed. You would need to do this by having two counters cascaded (i.e. connected in series) with each other. The overall block diagram is shown below.

The clktick_16 circuit generates a 1 cycle high pulse every 50,000 clock cycles. Therefore the output signal **tick** provides one enable pulse every 1 millisecond. (See Lecture 7 notes.)

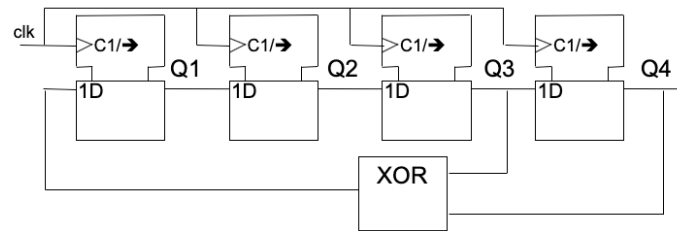


Modify your circuit to implement this and test the new circuit on the DE10 board.

² Synopsis Delay Constraint (.sdc) files are standard formatted files introduced by Synopsis, a well-known company specializing on IC design CAD tools. With this, a designer can specify various timing constraints for the CAD tools the check against. Here we are only using this to define clock frequency.

Task 3: Linear Feedback Shift Register (LFSR) and PRBS

You would have encountered a 4-bit LFSR in Lecture 7, which implements the primitive polynomial: $1 + X^3 + X^4$. The circuit is given as:



You are now required to implement a 7-bit LFSR implementing the polynomial: $1 + X^3 + X^7$. Assuming that you initialize the shift register to 7'd1, work out manually the first 10 sequence values of the output sequence. (The output sequence should be 127 long without repetition, is known as a **pseudo-random binary sequence** or **PRBS**.)

Connect the shift register clock to KEY[1] and use the momentary key to cycle through the first 10 or 20 values of the PRBS sequence. The "random" output should be displayed as two hexadecimal digits.